BOOKNEST

A PROJECT REPORT

Submitted by

ASMIT NAYAN [RA2411030030065]

KANHAIYA KUMAR [RA2411030030068]

SARTHAK SRIVASTAVA [RA2411030030047]

Under the guidance of

Dr. Anirudh Gaikwad

(Associate Professor, Department of Computer Science & Engineering) ***in partial fulfillment for the award of the degree***

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE & TECHNOLOGY, NCR CAMPUS

APRIL 2026

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this report titled "**BOOKNEST**" is the bonafide work of "**ASMIT NAYAN [Reg No: RA2411030030065], KANHAIYA KUMAR [Reg No: RA2411030030068], SARTHAK SRIVASTAVA [Reg No: RA2411030030047],** who carried out this work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Anirudh Gaikwad
Faculty Incharge
Associate Professor
Dept. of Computer Sciene & Engineering.

SIGNATURE

Dr. Neelam Singh
HEAD OF THE DEPARTMENT
Dept. of Computer Science & Engineering.

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our guide, Dr. Amit Kukker, Associate Professor, Department of Computer Science and Engineering, for her invaluable guidance, motivation, and constant encouragement throughout this project. We also extend our thanks to Dr. Neelam Singh, Head of the Department, for providing us with the necessary facilities and a conducive environment for our work.

We are grateful to SRM Institute of Science & Technology, NCR Campus, for the opportunity to undertake this project and to all faculty members who have helped us directly or indirectly. Finally, we thank our teammates for their cooperation and dedication throughout the project development.

ABSTRACT

BookNest is a web-based, database-driven application designed to automate and streamline the process of book lending and community-based reading management. The system addresses a common real-world problem faced by libraries and reading communities — the reliance on manual, paper-based record keeping that is slow, error-prone, and difficult to scale. By replacing these traditional methods with a centralized digital platform, BookNest provides a reliable single source of truth for all book, member, and lending data.

The system is built on a modern and production-grade technology stack. The backend is developed using FastAPI, a high-performance Python web framework, with SQLAlchemy serving as the ORM layer for clean and efficient database interaction. PostgreSQL is used as the primary relational database, storing all application data across four normalized tables — books, members, lendings, and reviews — with foreign key constraints enforcing referential integrity throughout. The frontend is built using pure HTML, CSS, and JavaScript, keeping the interface lightweight, stateless, and directly connected to the backend through RESTful API calls.

Key features of the system include a searchable book catalog with real-time availability tracking, a member registration and management module, a lending engine that handles book issuance and returns with automatic due-date calculation, and a community review system that allows members to rate and review books they have read. A live dashboard provides administrators with a quick overview of total books, active members, ongoing loans, and overdue returns.

The main objective of this project is to demonstrate a complete, working DBMS solution — from relational schema design and ORM mapping to REST API development and a functional web interface. Through the development of BookNest, practical knowledge of database normalization, API design, and full-stack integration has been gained, along with a deeper understanding of how real-world library management systems are built and maintained.

TABLE OF CONTENTS

Chapters.	Topic	Page No.
1	Introduction	6
2	Project Description	7-8
3	System Implementation	9-13
4	Results	14-15
5	Conclusion	16
6	References	17

Chapter 1 : INTRODUCTION

➤ **Main Idea:**

Our project is **BookNest** a full-stack, database-driven platform built with FastAPI, PostgreSQL, SQLAlchemy, and a pure HTML/CSS/JS frontend. It provides a complete solution for managing book lending and building a reading community.

The system includes:
• Book Catalog Management
• Member Registration and Tracking
• Lending Engine with Due-Date Tracking
• Community Reviews and Ratings
• REST API with Full CRUD Support

The BookNest is an important real-world application in the field of web development and database management. It helps in understanding how to design systems that handle real-time data, user interactions, and business operations efficiently.

This project allows members to borrow books, check availability, and write reviews. The admin can manage the book catalog, monitor lending activity, and keep records of all transactions.

The main objective of this project is to create a simple, efficient, and user-friendly platform that automates the traditional book lending process and reduces manual work.

By developing this system, we gain practical knowledge of:
• Web application development with FastAPI
• Relational database design and management with PostgreSQL
• ORM-based data access using SQLAlchemy
• RESTful API design and integration

This project is useful for understanding how real-world library systems operate and gives hands-on experience in building scalable and efficient database-driven applications.

Chapter 2 : PROJECT DESCRIPTION

The **BookNest** is designed to automate the process of managing book lending using a simple and interactive web-based interface. The application runs in a web browser and manages all library operations through a RESTful API built with FastAPI and PostgreSQL.

➤ System Overview

The system allows users to:
• Register and log in to the system
• Browse the available book catalog
• Check book details and availability status
• Borrow books and track the lending period
• View borrowing history and submit reviews

➤ Modules:

1. Input Module

o Accepts member details, book selection, and borrowing dates

2. User Module

o Handles member registration and profile management

3. Book Management Module

o Displays available books and manages the book catalog

4. Lending Module

o Processes lending and return requests and stores all transaction records

5. Admin Module

o Allows admin to manage the book catalog, members, and lending records

6. Payment Module

o Manages community reviews and star ratings for books

➤ Functional Requirements:

• Allow member registration and management
• Display available books with details and availability
• Enable book lending and returns
• Store and manage all lending records
• Provide a RESTful API for all CRUD operations

➤ Non-Functional Requirements:

• User-friendly interface
• Fast system response
• Secure data handling
• Reliable and scalable system

➤ Hardware and Software Requirements:

Software:

• Web Browser (Chrome/Edge/Firefox)
• HTML, CSS, JavaScript (Frontend)
• FastAPI with Python (Backend)
• PostgreSQL (Database) + SQLAlchemy (ORM)

Hardware:

• Minimum 4GB RAM
• Any modern computer or laptop

➤ Key Technologies and Design Aspects:

• System Implementation Language: Python, JavaScript
• Frontend: HTML, CSS, JavaScript
• Backend: FastAPI (Python)
• Database: PostgreSQL

Concepts Used:

o CRUD Operations (Create, Read, Update, Delete)
o ORM Mapping with SQLAlchemy
o RESTful API Design
o Three-Tier Architecture (Presentation, Application, Data)

Design:

Modular, minimal, and user-friendly interface with clean REST API endpoints and efficient data handling

Chapter 3 : SYSTEM IMPLEMENTATION

➤ System Architecture

BookNest follows a clean 3-tier architecture that separates the UI, business logic, and data storage:

- Presentation Layer — Pure HTML, CSS, and JavaScript frontend. Runs in the browser and communicates with the backend through REST API calls.
- Application Layer — FastAPI (Python) handles all routing, request validation via Pydantic schemas, and business logic such as availability checks.
- Data Layer — PostgreSQL stores all persistent data. SQLAlchemy ORM manages table mappings and database sessions.

All communication between the frontend and backend happens through REST API calls, keeping the UI fully decoupled from the data layer.

➤ Database Design

BookNest uses four normalized relational tables:

1. books

- book_id (PK), title, author, genre, isbn, published_year, total_copies, available_copies, description

2. members

- member_id (PK), name, email, phone, joined_on

3. lendings

- lending_id (PK), book_id (FK), member_id (FK), issued_on, due_date, returned_on

4. reviews

- review_id (PK), book_id (FK), member_id (FK), rating (1–5), comment, reviewed_on

Foreign key constraints enforce referential integrity across all four tables. One book can have many lending records. One member can borrow multiple books. Each review links a member to a specific book.

➤ Sample Code

database.py — PostgreSQL Connection Setup

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://postgres:asmit@localhost:5432/booknest"
)

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

schemas.py — Pydantic Request / Response Models

```
from pydantic import BaseModel
from typing import Optional, List
from datetime import date

class BookCreate(BaseModel):
    title: str
```

```
    author: str
    genre: Optional[str] = None
    isbn: Optional[str] = None
    total_copies: int = 1

class Book(BookCreate):
    id: int
    available_copies: int
    class Config:
        from_attributes = True

class LendingCreate(BaseModel):
    book_id: int
    member_id: int

class Lending(BaseModel):
    id: int
    book_id: int
    member_id: int
    issued_on: date
    due_date: date
    returned_on: Optional[date] = None
    class Config:
        from_attributes = True
```

main.py — FastAPI Routes

```
from fastapi import FastAPI, HTTPException, Depends
from sqlalchemy.orm import Session
from typing import List, Optional
import models, schemas, crud
from database import SessionLocal, engine

app = FastAPI(title="BookNest API", version="1.0.0")

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# -- Books --
@app.get("/books", response_model=List[schemas.Book])
def list_books(search: Optional[str] = None, db: Session = Depends(get_db)):
    return crud.get_books(db, search)

@app.post("/books", response_model=schemas.Book)
def add_book(book: schemas.BookCreate, db: Session = Depends(get_db)):
    return crud.create_book(db, book)

# -- Lending --
@app.post("/lendings", response_model=schemas.Lending)
def issue_book(lending: schemas.LendingCreate, db: Session =
Depends(get_db)):
    book = crud.get_book(db, lending.book_id)
    if not book:
        raise HTTPException(404, "Book not found")
    if book.available_copies < 1:
        raise HTTPException(400, "No copies available")
    return crud.create_lending(db, lending)
```

```
@app.patch("/lendings/{lending_id}/return", response_model=schemas.Lending)
def return_book(lending_id: int, db: Session = Depends(get_db)):
    lending = crud.return_book(db, lending_id)
    if not lending:
        raise HTTPException(404, "Lending record not found")
    return lending

# -- Reviews --
@app.post("/books/{book_id}/reviews", response_model=schemas.Review)
def add_review(book_id: int, review: schemas.ReviewCreate,
               db: Session = Depends(get_db)):
    if not crud.get_book(db, book_id):
        raise HTTPException(404, "Book not found")
    return crud.create_review(db, book_id, review)

# -- Stats --
@app.get("/stats")
def get_stats(db: Session = Depends(get_db)):
    return crud.get_stats(db)
```

➤ API Endpoints

BookNest exposes a fully documented REST API via Swagger UI at /docs, auto-generated by FastAPI.

- GET /books — Retrieve the full book catalog with optional search.
- POST /books — Add a new book to the catalog.
- DELETE /books/{id} — Remove a book from the catalog.
- POST /members — Register a new member.
- POST /lendings — Issue a book to a registered member.
- PATCH /lendings/{id}/return — Mark a book as returned and update inventory.
- POST /books/{id}/reviews — Submit a star rating and written review.
- GET /stats — Retrieve dashboard statistics (total books, members, active loans, overdue).

The workflow illustrates how different components of BookNest work together to manage book lending and community interactions efficiently.

• Application Start:
When the application is launched, the HTML/CSS/JS frontend loads in the web browser. The system connects to the PostgreSQL database through FastAPI and

retrieves the available book catalog. All modules are initialized and ready for operations.

• Member Registration:
New members are registered through the system. Their profile details are stored in the members table and they become immediately eligible to borrow books.

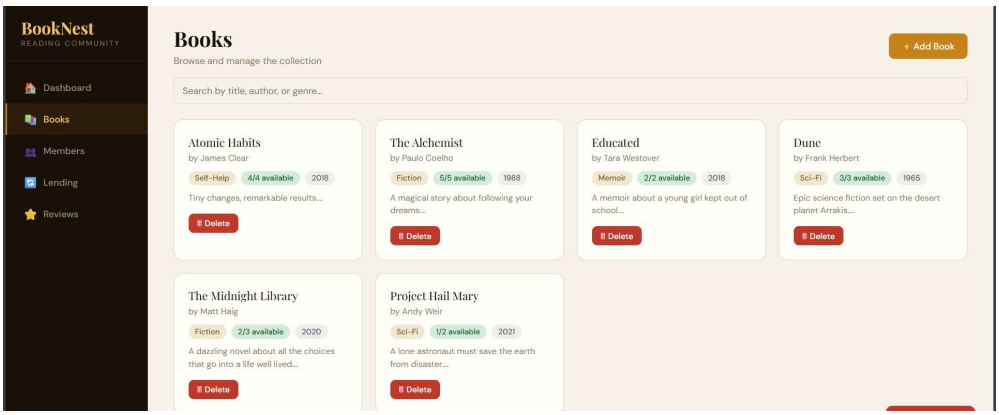

Fig 1. BookNest Dashboard — showing total books, members, active loans, reviews, and overdue count

• Book Browsing:
Users can browse the full book catalog. The system fetches book details such as title, author, genre, and availability from the database and displays them through the GET /books API endpoint.

• Book Selection:
The member selects a book to borrow. The system checks availability and records the issue date using the POST /lending endpoint.

• Lending Processing:
Once confirmed, the lending module stores the transaction in the Lendings table. The system automatically updates the book's availability status and creates a lending record for tracking.

• Admin Management:
The admin can add, update, or delete books, monitor active lendings, and manage member records. All actions go through the REST API and directly update the PostgreSQL database.

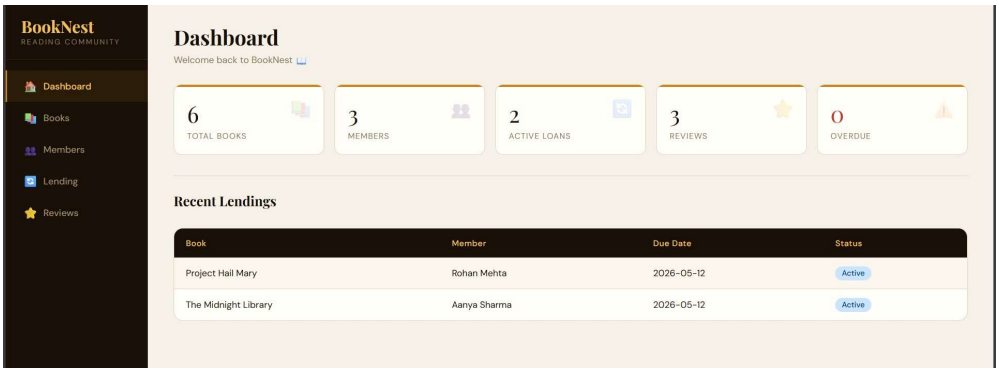

Fig 2. BookNest Book Catalog — searchable list with availability indicators and genre tags

• Result Display:
The system displays lending confirmation details including book name, member name, and due date. Members can also view their borrowing history and submit reviews.

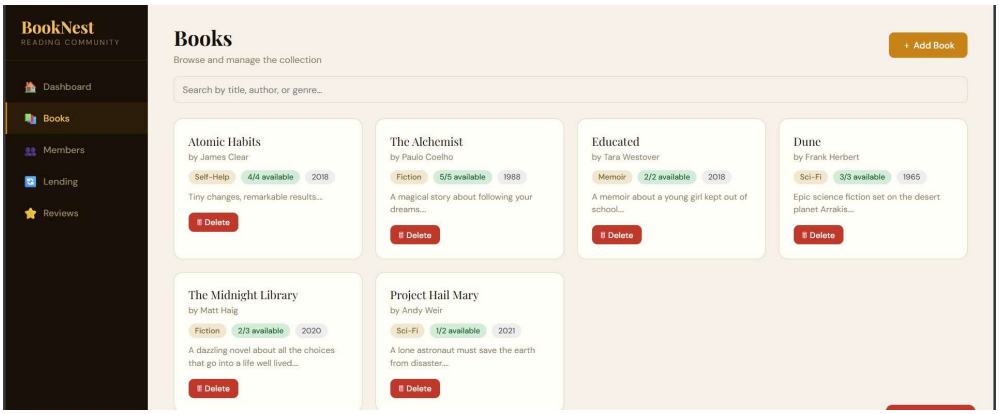

Fig 3. BookNest Recent Lendings — showing active loans with due dates and member names

• Review Submission:
Members can submit star ratings and written reviews for books they have read. These are stored in the Reviews table and linked to both the member and the book.

This workflow ensures smooth interaction between members, system modules, and the database, resulting in efficient and accurate library management.

Chapter 4: Results and Discussion

BookNest successfully achieved efficient management of book lending and member interactions.

• System Accuracy:
The system correctly handled all user inputs, validated lending details, and ensured that only available books could be borrowed. PostgreSQL foreign key constraints maintained consistency and accuracy of records.

• Performance Analysis:
The system demonstrated fast response times for requests such as browsing the catalog, issuing books, and submitting reviews. FastAPI's async capabilities and SQLAlchemy ORM kept database queries efficient.

• User Interface and Output:
The interface provided a clean and minimal design where users could easily navigate through options. Lending details, book information, and review results were displayed clearly through the API responses.

• Project Validation:
The project validates our understanding of DBMS concepts such as relational schema design, ORM mapping, RESTful API development, and frontend-backend integration. It demonstrates how a real-world library system can be built using modern and efficient technologies.

Chapter 5 : CONCLUSION

The BookNest project has been successfully developed and implemented.
It demonstrates how a real-world library service can be automated using modern web technologies and a well-designed database system.

The project helped us understand:
• Full-stack web application architecture
• Database management and connectivity
• ORM-based data access using SQLAlchemy
• RESTful API design and system integration

Through this project, we gained practical knowledge of Python, FastAPI, PostgreSQL, SQLAlchemy, and frontend technologies.
The system works efficiently and provides accurate booking and management functionality.

➤ **Future Enhancements:**

• JWT-based authentication for role-specific access (admin vs. member)
• Migration to a modern React or Vue frontend for better user experience
• ML-based book recommendation system using borrowing history and ratings
• Advanced search and filtering by title, author, genre, and availability
• Cloud deployment for scalability and wider access

Chapter 6 : REFERENCES

- FastAPI Official Documentation – https://fastapi.tiangolo.com
- SQLAlchemy Official Documentation – https://docs.sqlalchemy.org
- PostgreSQL Official Documentation – https://www.postgresql.org/docs
- GeeksforGeeks – Database Management Systems
- Lecture Notes (DBMS Subject)